

Near-Real-Time Financial Forecasting on AWS.

Tesla · Microsoft · Nvidia `Two horizons · fully serverless · eu-west-1`

TEAM 11_02

Ralph Khairallah · Ilias El Mariky · Dalibor Svonavec · Francesco Barillari · Arthur Bohin

FINAL PROJECT · 14 SLIDES · 12 MIN

Forecast three tickers, two horizons, fully automated.

TSLA, MSFT, NVDA · equity prices and live news, eu-west-1.

Hourly 1 h ahead, refreshed hourly, sentiment-aware.

Daily 7d ahead, refreshed once a day, prices only.

Event-driven and fully serverless · no servers to manage, scales to zero.

In scope: 3 tickers · 2 horizons · 1 dashboard · automation end-to-end.

Cut from first draft: wider ticker universe, custom news scraper.

REGION

eu-west-1

ACCOUNTS

4

SEPARATE AWS ACCOUNTS

LAMBDA

11

AWS SERVICES

12

END-TO-END FLOW

INPUTS

**Prices
News**

›

PIPELINE

Serverless AWS

11 Lambdas · Glue ·
DeepAR

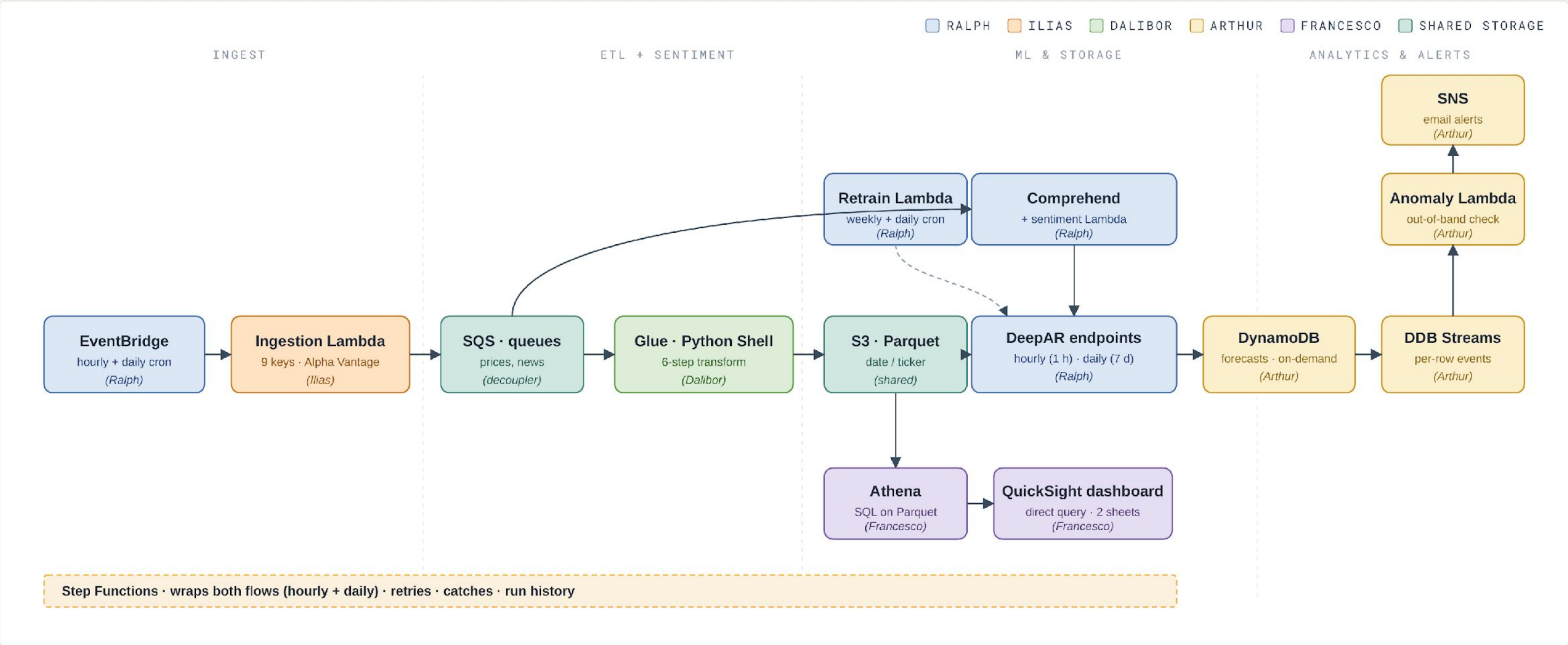
›

OUTPUTS

**Forecasts
+ Alerts**

→ QuickSight dashboard (hourly + daily)

End-to-end pipeline, colour-coded by owner



EventBridge → Lambda → SQS, with 9 rotating API keys.

▶	2026-05-24T16:25:01.445Z	[OK] NVDA price @ 215.33 sent to prices queue
▶	2026-05-24T16:25:02.968Z	[NEWS DEBUG] NVDA raw response keys: ['items', 'sentiment_score_definition', 'relevance_score_definition', 'feed']
▶	2026-05-24T16:25:03.125Z	[OK] NVDA news (5 articles) sent to news queue
▶	2026-05-24T16:25:03.127Z	END RequestId: c54c09f0-8ea4-459f-a5c0-8f55a6583fe4
▶	2026-05-24T16:25:03.127Z	REPORT RequestId: c54c09f0-8ea4-459f-a5c0-8f55a6583fe4 Duration: 6026.28 ms Billed Duration: 6505 ms Memory Size: 128 MB Max Memory Used: 88 MB Init Duration: 478.24 ms

Étape 2 : élaborer la planification

Planification d'événement

Infos

Fréquence fixe de

1 hour

Modifier

Étape 3 : sélectionnez la ou les cibles

Cibles

informations	Nom de la cible	Type	ARN	Entrée	Rôle
▼	ccbda-group11-02-data-ingestion	Fonction Lambda	 <code>arn:aws:lambda:eu-west-1:628454062331:function:ccbda-group11-02-data-ingestion</code>	Événement correspondant	-

Entrée à la cible :

Paramètres supplémentaires :

File d'attente de lettres mortes (DLQ) :

Événement correspondant

--

-

Modifier

EventBridge schedule fires **hourly** for prices and news (offset crons).

One Lambda calls **Alpha Vantage** (prices + news) and publishes to two SQS queues.

9 API keys rotated in-flight · Alpha Vantage free tier is 25 req/day/key.

Messages go to 2 SQS queues (one for prices, one for news)

CADENCE

Hourly

API KEYS

9

TICKERS

3

Glue Python Shell · SQS messages → partitioned Parquet.

SIX-STEP TRANSFORM

- 01
- Drain SQS price messages (JSON)
- 02
- Drop duplicates on (ticker, timestamp) and round timestamps
- 03
- Align to a common 5-min time grid and gap fill
- 04
- Join price + sentiment per ticker
- 05
- Type-cast and sort
- 06
- Write Parquet · partition by ticker

ENGINE

Python Shell

COLD-START

OUTPUT

Parquet

TICKER PARTITIONED

ON FAILURE

CW alarm

```
{
  "item_id": "MSFT",
  "timestamp": "2026-05-14T20:05:00Z",
  "target_value": 405.21,
  "sentiment_score": 0.0002
}
{
  "item_id": "MSFT",
  "timestamp": "2026-05-14T20:10:00Z",
  "target_value": 405.21,
  "sentiment_score": 0.0002
}
{
  "item_id": "MSFT",
  "timestamp": "2026-05-14T20:15:00Z",
  "target_value": 405.21,
  "sentiment_score": 0.0002
}
{
  "item_id": "MSFT",
  "timestamp": "2026-05-14T20:20:00Z",
  "target_value": 405.21,
  "sentiment_score": 0.0002
}
{
  "item_id": "MSFT",
  "timestamp": "2026-05-14T20:25:00Z",
  "target_value": 409.43,
  "sentiment_score": -0.0061
}
{
  "item_id": "MSFT",
  "timestamp": "2026-05-14T20:30:00Z",
  "target_value": 409.43,
  "sentiment_score": -0.0061
}
{
  "item_id": "MSFT",
  "timestamp": "2026-05-14T20:35:00Z",
  "target_value": 409.43,
  "sentiment_score": -0.0061
}
{
  "item_id": "MSFT",
  "timestamp": "2026-05-14T20:40:00Z",
  "target_value": 409.43,
  "sentiment_score": -0.0061
}
```

team01-data-preparation

Last modified on 13. 5. 2026 13:35:12

Actions Save Run

Script

Job details

Runs

Data quality

Schedules

Version Control

Job runs (1/138)

Info

Last updated (UTC) May 19, 2026 at 15:41:15

View details

Stop job run

Troubleshoot with AI

Table View

Card View

Filter job runs by property

	Run status	Retries	Start time (Local)	End time (Local)	Duration	Capacity (DPUs)	Worker type	Glue version
	Succeeded	0	05/19/2026 16:42:47	05/19/2026 16:43:51	35 s	0.0625 DPUs	-	3.0
	Succeeded	0	05/19/2026 15:42:47	05/19/2026 15:43:17	23 s	0.0625 DPUs	-	3.0
	Succeeded	0	05/19/2026 14:42:47	05/19/2026 14:43:14	21 s	0.0625 DPUs	-	3.0
	Succeeded	0	05/19/2026 13:42:46	05/19/2026 13:43:31	36 s	0.0625 DPUs	-	3.0
	Succeeded	0	05/19/2026 12:42:47	05/19/2026 12:43:29	34 s	0.0625 DPUs	-	3.0

Run details

Input arguments (8)

Logs

Run insights

Metrics

Troubleshooting analysis

Spark UI

Job name

team01-data-preparation

Id

jr_6f733a40c81c7eadbca0cb0a62b78abfca49cc5d5b74353a05/19/2026 16:43:51

Run status

Succeeded

Retry attempt number

-

Initial run

Trigger name

-

Start time (Local)

05/19/2026 16:42:47

End time (Local)

05/19/2026 16:43:51

Start-up time

29 seconds

Execution time

35 seconds

Security configuration

-

Glue version

3.0

Worker type

-

Max capacity

0.0625 DPUs

Execution class

-

Cloudwatch logs

Output logs

Error logs

Last modified on (Local)

05/19/2026 16:43:51

Log group name

/aws-glue/python-jobs

Number of workers

-

Timeout

30 minutes

Usage profile

-

2026/05/19 15:42:47,

Rewind job b

Job name

team01-data-preparation

Id

jr_bd98df0617a27343c620fc43f52561dfe054672b222e926858aaca6d1e69573a

Run status

Succeeded

Glue version

3.0

Retry attempt number

Initial run

Start time (Local)

05/19/2026 17:42:47

End time (Local)

05/19/2026 17:43:10

Start time (UTC)

2026/05/19 15:42:47

End time (UTC)

2026/05/19 15:43:10

Start-up time

6 seconds

Execution time

17 seconds

Last modified on (Local)

05/19/2026 17:43:10

Last modified on (UTC)

2026/05/19 15:43:10

Trigger name

-

Security configuration

-

Timeout

30 minutes

Max capacity

0.0625 DPUs

Number of workers

-

Worker type

-

Execution class

-

Log group name

/aws-glue/python-jobs

Cloudwatch logs

Output logs

Error logs

Performance and debugging recommendations

View in CloudWatch

Usage profile

-

Input arguments (8)

News → Comprehend → a signed score, fed to the hourly model.

SQS on proj1102-news-queue **triggers** the sentiment Lambda (event-source mapping).

Lambda calls **Amazon Comprehend** per headline.

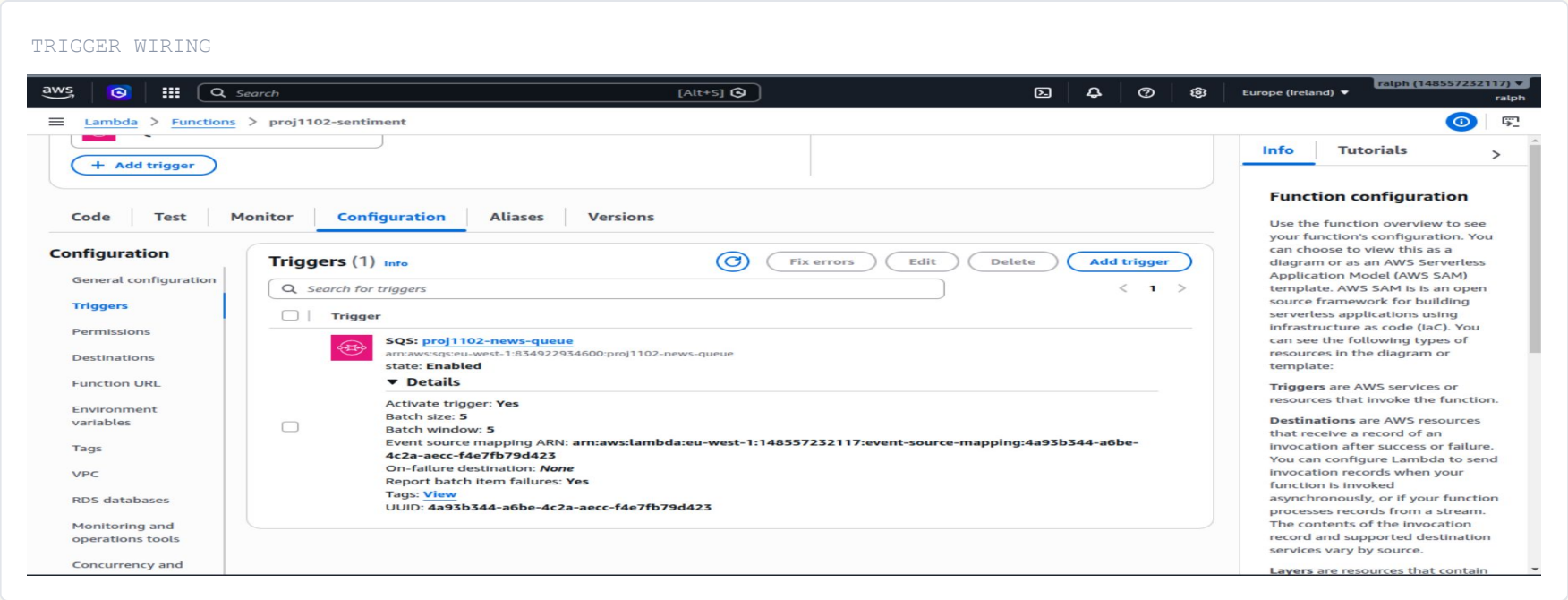
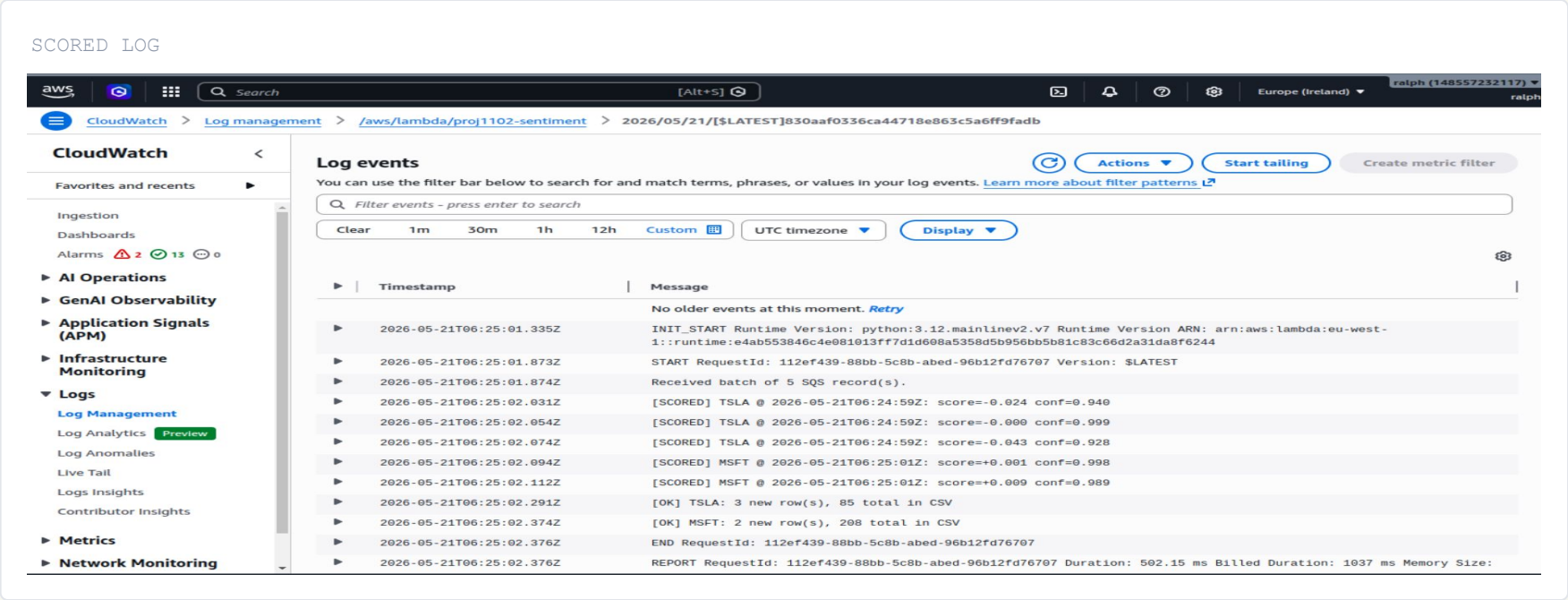
Probabilities collapsed into a **signed score** in [-1, +1].

Written back as a **related time series** next to price.

Used **only** by the hourly model · daily is prices-only.

Every run logged · CloudWatch monitor on invocation count.

```
score = Ppositive - Pnegative · range [-1, +1]
```



One global model, probabilistic output, serverless endpoints.

SageMaker DeepAR · deep auto-regressive RNN.

One **global** model learned jointly across all tickers.

Output is a **distribution**, not a point · P10 / P50 / P90.

Hourly endpoint takes **sentiment** as a related series.

Serverless endpoints · scale to zero between calls.

Sub-second invocation on warm path · bursty workload.

ENDPOINTS

2

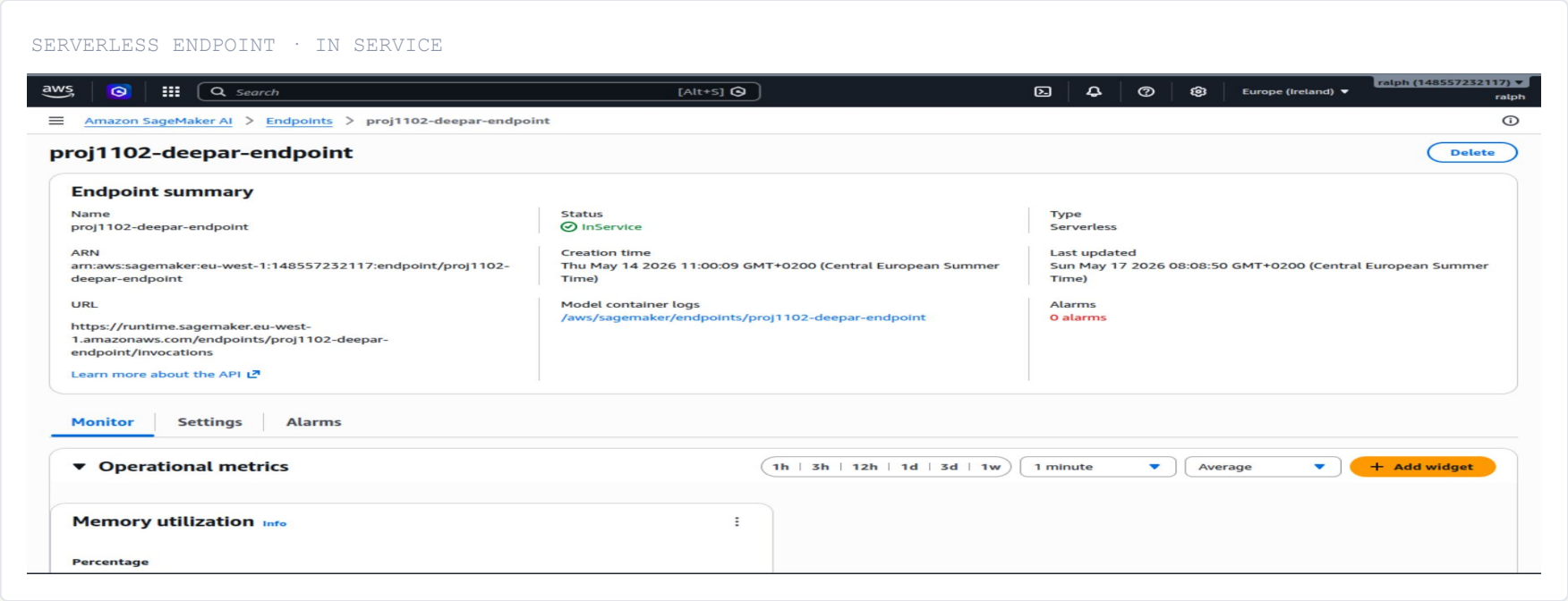
HOURLY + DAILY

OUTPUT

P10/50/90

IDLE COST

≈ \$0



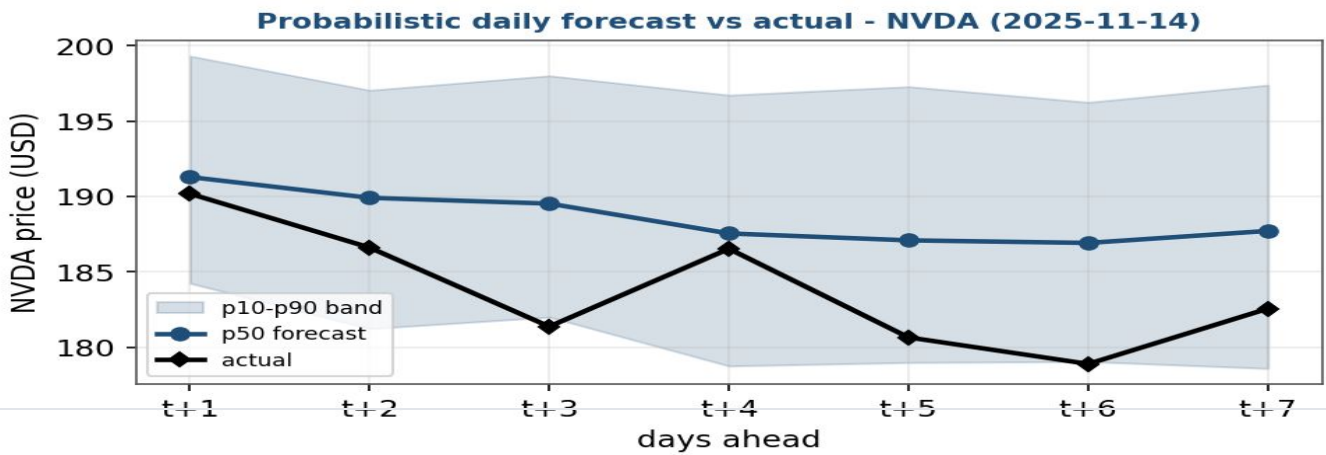
Same components, two configurations.

HOURLY TRACK	
1 hour ahead refreshed every hour	
CADENCE	every hour
LOOKAHEAD	1 hour
INPUTS	prices + sentiment
MODEL	DeepAR hourly endpoint
RETRAIN	weekly cron (Sun 06:00)
USED FOR	intraday reaction

DAILY 7-DAY FAN · THE BAND WIDENS WITH HORIZON

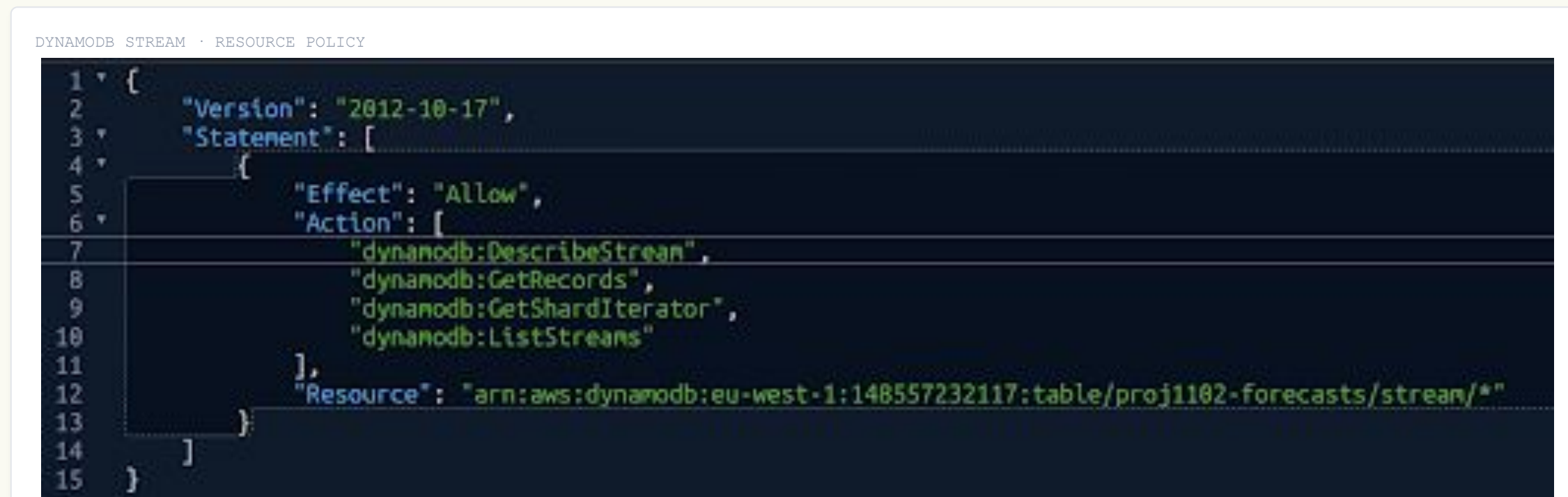
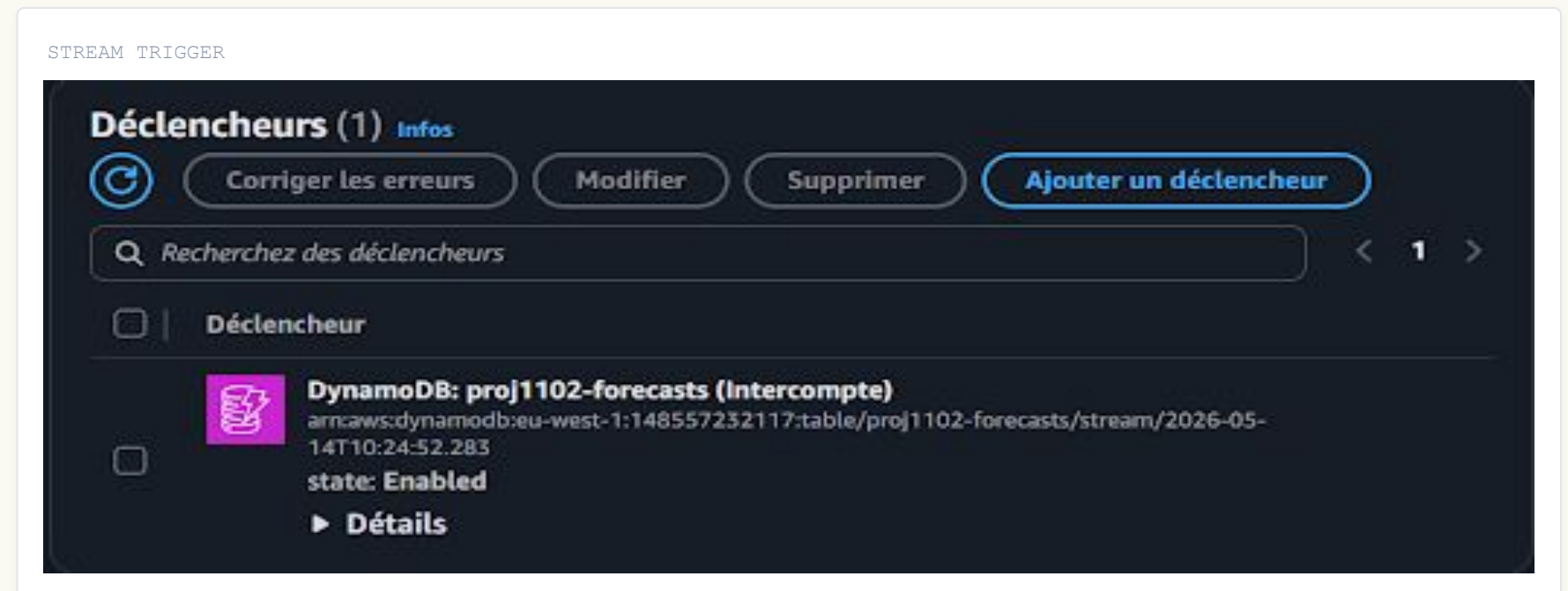
Same DeepAR algorithm, different horizons. Hourly tracks the market; daily surfaces trend + uncertainty.

DAILY TRACK	
7 days ahead refreshed once a day	
CADENCE	once a day
LOOKAHEAD	7 days
INPUTS	prices only
MODEL	DeepAR daily endpoint
RETRAIN	daily cron
USED FOR	short-term trend

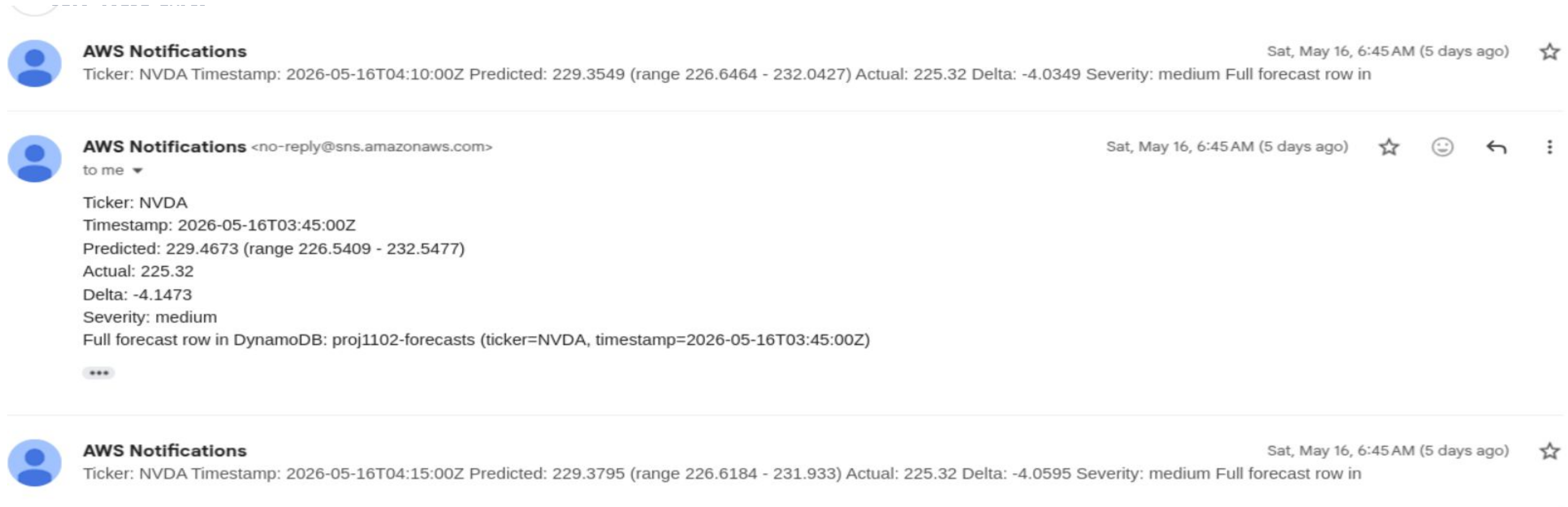


DynamoDB → Streams → anomaly Lambda → SNS email.

1. **DynamoDB** · on demand · pay-per-write, no capacity to manage.
2. Look at the section **is_anomaly** for each data.
3. If true, then extract all the **financial data** of the entry (p50, p10, p90, ticker, actual).
4. **Send a SNS alert** to every group members with all the data inside.



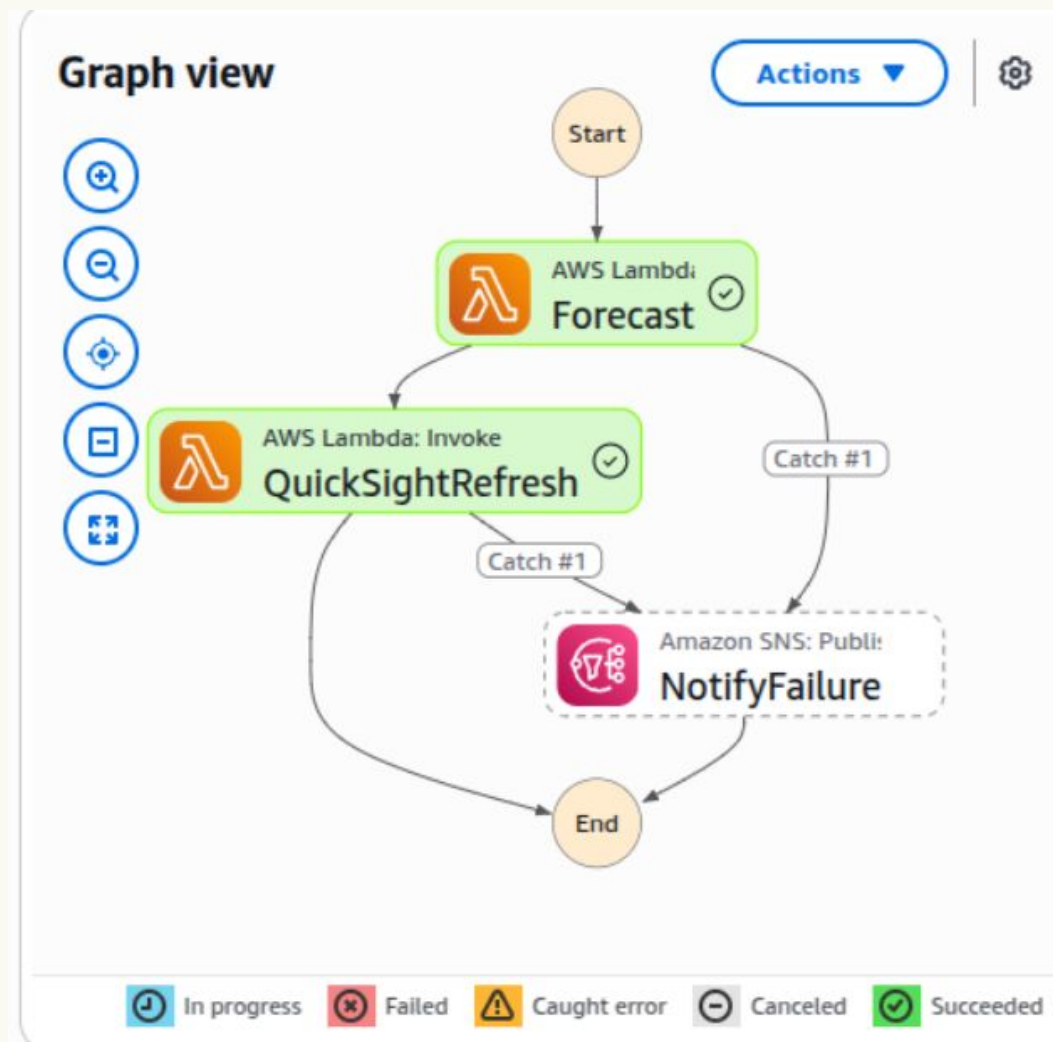
Alert Example



Step Functions : Orchestration of the pipeline

- **Organisation** : Chain AWS services (Lambda, Glue, Forecast)
- **Resilience** : error handling (Retry and Catch mechanism)
- **Visibility** : Track executions, identify problems (visual interface)

Graph View :



Steps :

1. **Design** : Define logical steps of the pipeline
2. **Connect** : Link all AWS services
3. **Logic** : Manage conditional branching and errors
4. **Deployment** : Launch and monitor the process in real time

Really useful to deal with cross accounts !

Athena → QuickSight : one sheet per horizon

S3: stores actual, sentiment and predictions' values

Athena: executes SQL queries on unified views

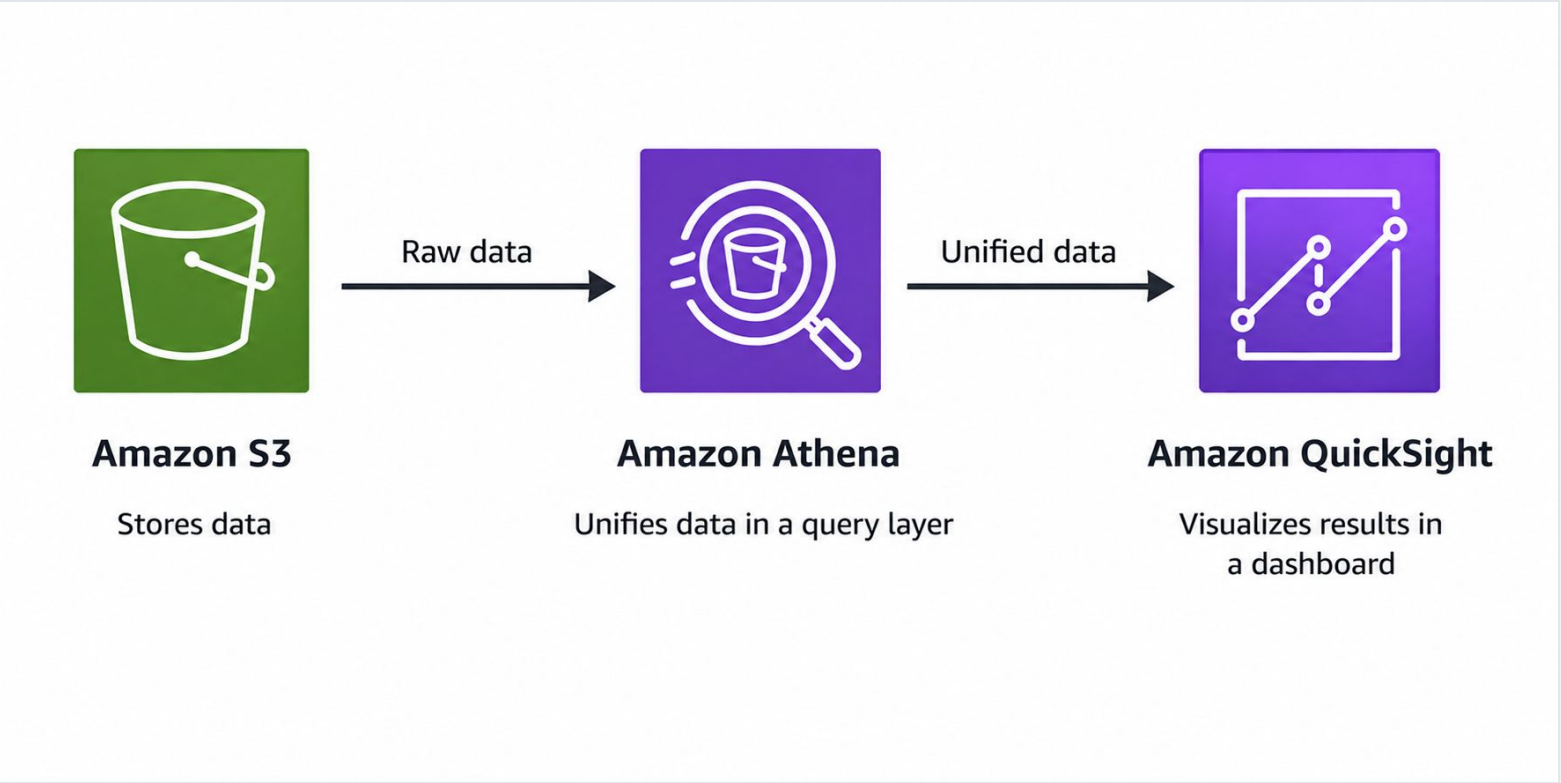
QuickSight: runs live Athena queries and visualizes interactive forecasting analytics

Sheet 1 · **Hourly** · 1-hour-ahead forecasts

Sheet 2 · **Daily** · 7-day-ahead forecasts

Interactive dashboard thanks to **URL** and **navigation** action

TSLA - NVDA - MSFT analysed in both sheets

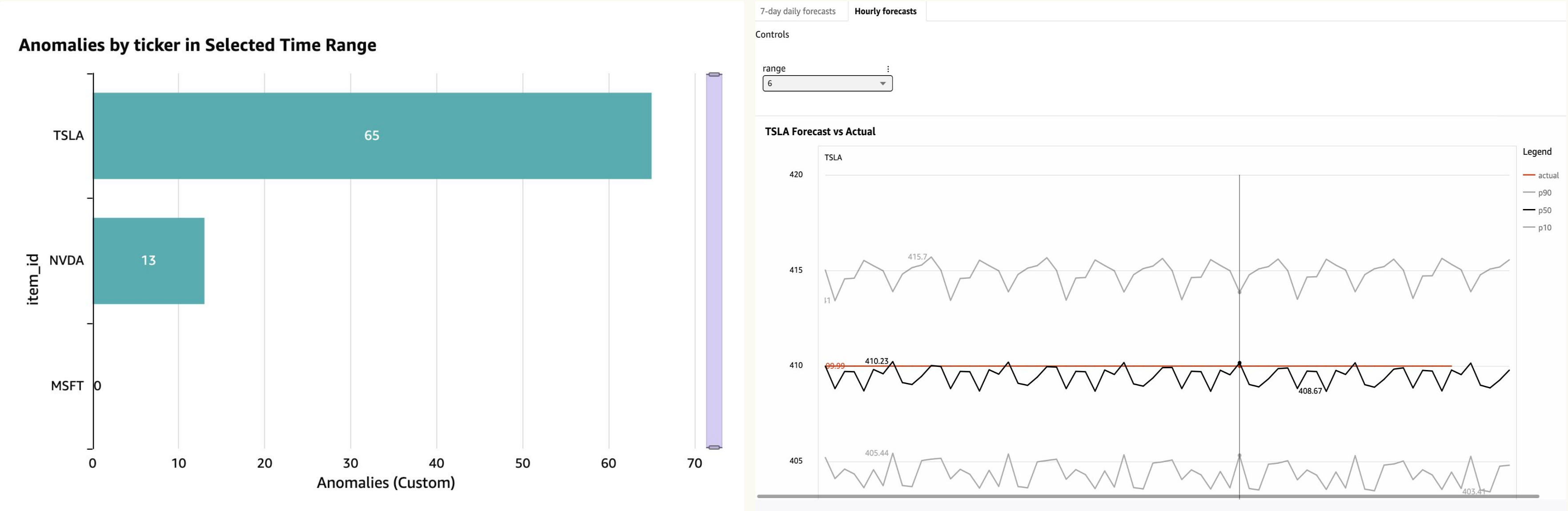


Latest actual value by ticker

TICKER	DATE	VALUE
MSFT	May 19, 2026	423.54
NVDA	May 19, 2026	222.32
TSLA	May 19, 2026	409.99

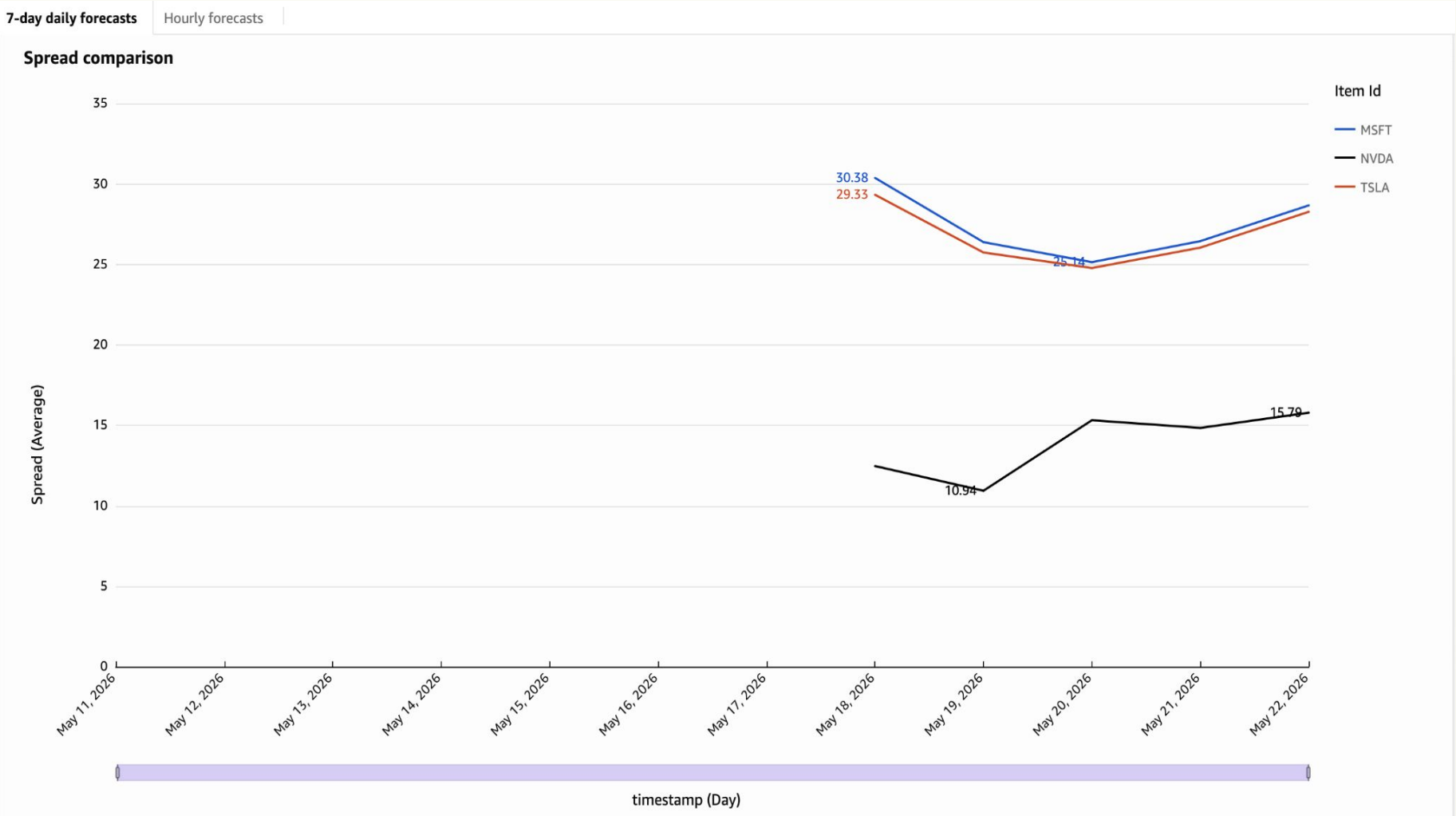
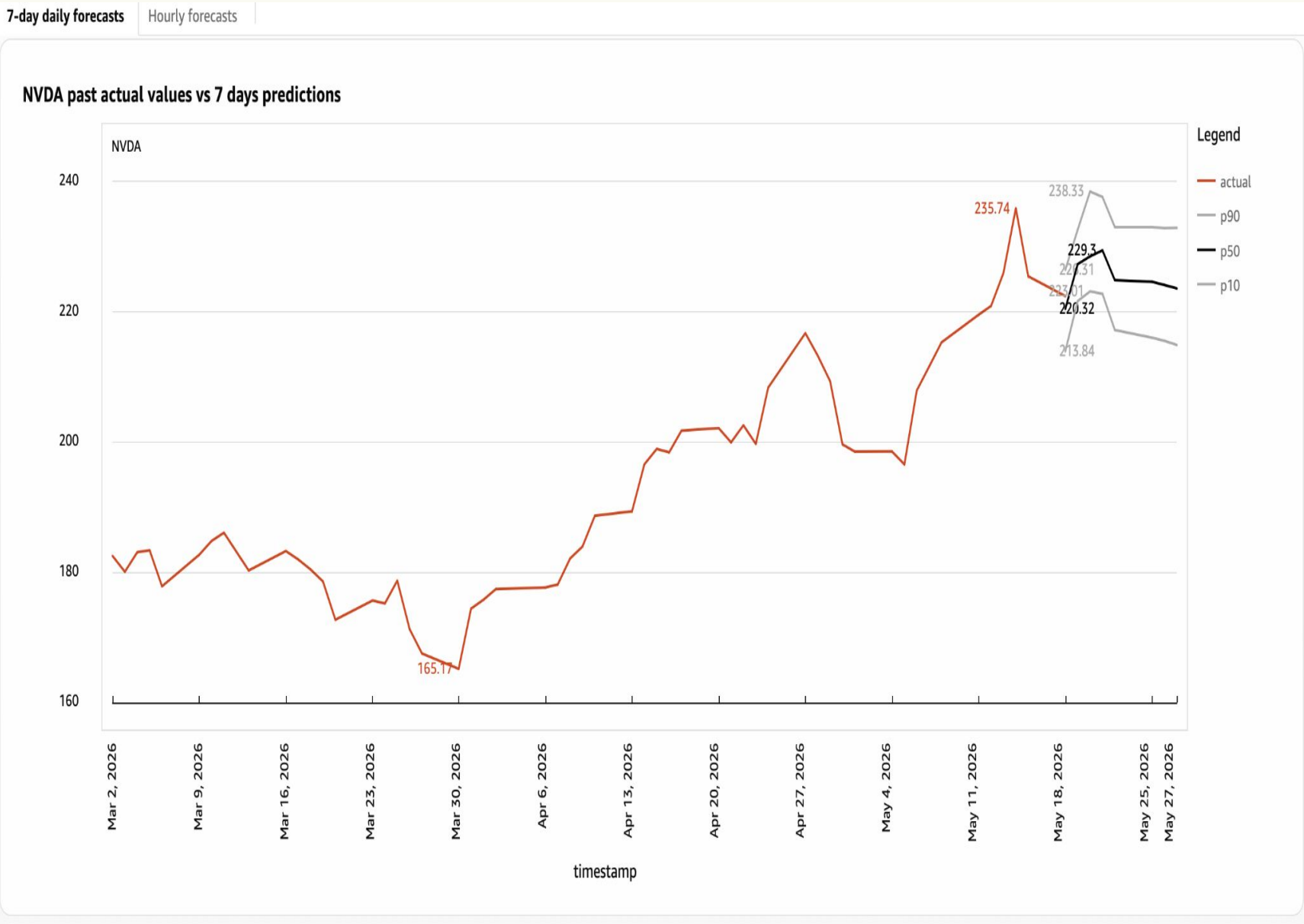
Hourly Forecasts sheet

- Short-range predictions with a forecasting horizon of one hour ahead, updated at 5-minute intervals.
- P50 vs actuals, P10–P90 band, anomaly counter, MAPE, spread chart, forecast bias indicator, time range, latest value.



7-day Daily Forecasts sheet

- Longer-range predictions with a forecasting horizon of 7 days ahead, updated once a day.
- Spread comparison chart across the three assets.



Engineering hygiene and what we learned.

ENGINEERING

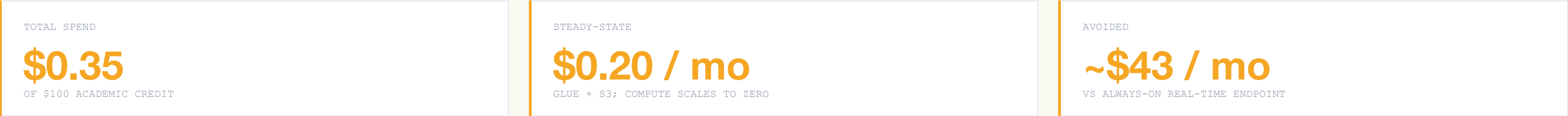
- GitHub Actions CI on every push · static checks + 50 pytest test .
- Docker health-probe container for the API surface.
- Makefile wraps every operator action (make check,make health ...)
- Twelve-factor · 10 fully met · 1 N/A (Port binding) · 1 partial (Dev/prod parity)



LESSONS

- Cross-account IAM was the hardest part · the streams policy in particular.
- Probabilistic validation only works on real data · the P90 story matured late.
- Pivoted away from AWS Forecast early · DeepAR gave us the control we needed.

COST · ACADEMIC CREDIT USAGE



Automated, serverless, end to end. Thank you. Questions?